

```
[USER/SUDO] dnf / dnf5
```

```
DESCRIPTION : Gestionnaire de paquets principal de Fedora/RHEL/CentOS (successeur de yum)
POURQUOI : Installer, mettre à jour, supprimer et rechercher des paquets RPM avec
           gestion automatique des dépendances. dnf5 est la réécriture en C++ –
           plus rapide, utilisée par défaut depuis Fedora 41.
CONTEXTE : Administration système, installation de logiciels, mises à jour.
SOUS-COMMANDES PRINCIPALES :
  install PAQUET      Installe un ou plusieurs paquets
  remove PAQUET       Supprime un paquet et ses dépendances orphelines
  update / upgrade    Met à jour tous les paquets (ou un paquet spécifique)
  downgrade PAQUET    Rétrograde un paquet vers une version antérieure
  reinstall PAQUET    Réinstalle un paquet
  search MOT          Recherche dans les noms et descriptions
  info PAQUET         Affiche les informations détaillées d'un paquet
  list                Liste les paquets (installed, available, updates...)
  provides FICHIER     Trouve quel paquet fournit un fichier ou commande
  repolist            Liste les dépôts actifs
  repoquery           Interroge les métadonnées des paquets
  history             Historique des transactions
  autoremove          Supprime les paquets orphelins inutilisés
  clean all           Vide le cache des métadonnées et paquets
  makecache           Télécharge les métadonnées des dépôts
  check-update        Liste les mises à jour disponibles (sans installer)
  distro-sync         Synchronise avec les versions du dépôt
  group install GROUPE Installe un groupe de paquets
  group list          Liste les groupes disponibles
  module list         Liste les modules AppStream
  module install MODULE:STREAM Installe un flux de module
  config-manager       Gère la configuration des dépôts
OPTIONS GLOBALES :
  -y                 Répond oui automatiquement
  -q                 Mode silencieux
  --nogpgcheck       Ignore la vérification GPG
  --disablerepo=NOM  Désactive un dépôt pour cette commande
  --enablerepo=NOM   Active un dépôt pour cette commande
  --exclude=PAQUET  Exclut un paquet de la transaction
  --downloadonly     Télécharge sans installer
  --downloadaddir=DIR Répertoire de téléchargement
  --best             Installe la meilleure version disponible
  --allowerasing     Autorise la suppression de paquets pour résoudre les dépendances
  --skip-broken      Ignore les paquets problématiques
  --setopt=OPTION=VALEUR Définit une option de configuration temporaire
EXEMPLES :
$ sudo dnf install vim git curl
$ sudo dnf update
$ sudo dnf remove paquet
$ dnf search "éditeur de texte"
$ dnf info firefox
$ dnf provides /usr/bin/python3
$ dnf repoquery --requires vim
$ dnf repoquery --whatprovides /etc/passwd
$ sudo dnf install --downloadonly --downloadaddir=/tmp/paquets vim
$ sudo dnf history
$ sudo dnf history undo 5
$ sudo dnf autoremove
$ sudo dnf clean all
$ sudo dnf config-manager --add-repo https://repo.example.com/fedora.repo
$ sudo dnf config-manager --set-enabled nomrepo
```

```
[USER] dnf4 / dnf-3
```

```
DESCRIPTION : Versions spécifiques de DNF – dnf4 est l'ancienne génération Python,
              dnf-3 est un alias de compatibilité
POURQUOI : Compatibilité avec des scripts ou plugins écrits pour une version
           précise de DNF.
CONTEXTE : Scripts legacy, transition dnf4 → dnf5.
EXEMPLES :
```

```
$ dnf4 install paquet
$ dnf-3 search paquet
```

```
[USER] yum / microdnf
```

DESCRIPTION : yum : prédécesseur de DNF – alias vers dnf sur les systèmes modernes
microdnf : version légère de DNF pour les conteneurs

POURQUOI : Compatibilité avec les scripts anciens (yum). microdnf est utilisé dans les images conteneurs minimales sans dnf complet.

CONTEXTE : Scripts legacy (yum), images conteneurs minimales (microdnf).

EXEMPLES :

```
$ sudo yum install paquet
$ sudo microdnf install vim
$ sudo microdnf update
```

```
[USER] rpm
```

DESCRIPTION : Gestionnaire de paquets RPM de bas niveau – installe, interroge, vérifie et supprime des paquets .rpm directement

POURQUOI : Opérations de bas niveau sur les paquets RPM sans passer par DNF. Interroger la base de données RPM, vérifier l'intégrité des paquets, extraire des informations détaillées.

CONTEXTE : Administration avancée, scripts, forensique, vérification d'intégrité.

OPTIONS PRINCIPALES :

-i / --install FICHIER.rpm	Installe un paquet [SUDO]
-U / --upgrade FICHIER.rpm	Met à jour (ou installe si absent) [SUDO]
-F / --freshen FICHIER.rpm	Met à jour seulement si déjà installé [SUDO]
-e / --erase PAQUET	Supprime un paquet [SUDO]
-q / --query	Mode interrogation
-V / --verify	Mode vérification
-K / --checksig	Vérifie la signature d'un fichier .rpm

OPTIONS DE REQUÊTE (-q) :

-qa	Liste tous les paquets installés
-qi PAQUET	Informations détaillées sur un paquet
-ql PAQUET	Liste les fichiers d'un paquet installé
-qc PAQUET	Liste les fichiers de configuration
-qd PAQUET	Liste les fichiers de documentation
-qR PAQUET	Liste les dépendances (Requires)
-q --provides PAQUET	Ce que le paquet fournit
-qf FICHIER	Quel paquet fournit ce fichier
-qp FICHIER.rpm	Interroge un fichier .rpm (pas encore installé)
--changelog PAQUET	Affiche le changelog
--scripts PAQUET	Affiche les scripts pre/post install
--queryformat FORMAT	Format de sortie personnalisé

OPTIONS D'INSTALLATION :

-v	Mode verbeux
-h	Affiche une barre de progression (#)
--nodeps	Ignore les dépendances (DANGER)
--force	Force l'installation
--test	Simulation sans installer
--noscripts	N'exécute pas les scripts
--nosignature	Ignore la vérification de signature

EXEMPLES :

```
$ rpm -qa | grep python
$ rpm -qi firefox
$ rpm -ql vim
$ rpm -qf /usr/bin/vim
$ rpm -qR paquet.rpm
$ sudo rpm -ivh paquet.rpm
$ sudo rpm -Uvh paquet.rpm
$ sudo rpm -e paquet
$ rpm -V vim
$ rpm -Va 2>/dev/null | grep "^..5"
$ rpm -K paquet.rpm
$ rpm -qa --queryformat "%{NAME} %{VERSION} %{INSTALLTIME:date}\n" | sort
```

```
[USER] rpm2cpio / rpm2archive
```

DESCRIPTION : Convertit un fichier RPM en archive cpio ou tar

POURQUOI : Extraire les fichiers contenus dans un RPM sans l'installer – utile pour récupérer un fichier spécifique ou inspecter le contenu.

CONTEXTE : Récupération de fichiers, inspection de paquets, forensique.

```

EXEMPLES      :
$ rpm2cpio paquet.rpm | cpio -idv
$ rpm2cpio paquet.rpm | cpio -t
$ rpm2cpio paquet.rpm | cpio -id "/usr/bin/vim"
$ rpm2archive paquet.rpm
$ rpm2archive paquet.rpm && tar -tzf paquet.rpm.tgz

```

```
[SUDO] rpmdb
```

```

DESCRIPTION   : Gestion de la base de données RPM
POURQUOI      : Reconstruire ou vérifier la base de données RPM en cas de corruption.
CONTEXTE      : Réparation système, administration avancée.
OPTIONS       :
--rebuilddb    Reconstruit la base de données RPM
--initdb       Initialise une nouvelle base de données
--verifydb     Vérifie l'intégrité de la base de données
EXEMPLES      :
$ sudo rpmdb --rebuilddb
$ sudo rpmdb --verifydb

```

```
[SUDO] rpmkeys
```

```

DESCRIPTION   : Gestion des clés GPG de la base de données RPM
POURQUOI      : Importer, lister ou supprimer les clés GPG utilisées pour vérifier
                  l'authenticité des paquets RPM.
CONTEXTE      : Sécurité des paquets, ajout de dépôts tiers.
OPTIONS       :
--import FICHIER_CLÉ Importe une clé GPG
-K FICHIER.rpm      Vérifie la signature d'un RPM
--list-keys         Liste les clés importées
EXEMPLES      :
$ sudo rpmkeys --import /etc/pki/rpm-gpg/RPM-GPG-KEY-fedora-41-x86_64
$ rpmkeys -K paquet.rpm
$ rpmkeys --list-keys

```

```
[USER] rpmbuild
```

```

DESCRIPTION   : Construit des paquets RPM depuis des fichiers sources et un fichier .spec
POURQUOI      : Créer ses propres paquets RPM – packaging de logiciels pour Fedora/RHEL.
CONTEXTE      : Packaging RPM, contribution aux distributions, déploiement interne.
OPTIONS       :
-ba FICHIER.spec    Construit les paquets binaires ET source
-bb FICHIER.spec    Construit seulement le paquet binaire
-bs FICHIER.spec    Construit seulement le paquet source (SRPM)
-bp FICHIER.spec    Exécute seulement la phase %prep
-bc FICHIER.spec    Exécute jusqu'à la phase %build
-bi FICHIER.spec    Exécute jusqu'à la phase %install
--rebuild FICHIER.src.rpm Reconstruit un paquet depuis un SRPM
-D "MACRO VALEUR"   Définit une macro RPM
--noclean           Ne supprime pas le répertoire de build après
--nocheck           Passe la phase %check (tests)
--target ARCH       Architecture cible
STRUCTURE DU RÉPERTOIRE (~rpmbuild/) :
SPECS/              Fichiers .spec
SOURCES/            Sources (tarballs, patches)
BUILD/              Répertoire de compilation
RPMS/               Paquets binaires générés
SRPMS/              Paquets source générés
EXEMPLES          :
$ rpmdev-setuptree
$ rpmbuild -ba ~/rpmbuild/SPECS/monpaquet.spec
$ rpmbuild -bb SPECS/monpaquet.spec
$ rpmbuild --rebuild monpaquet.src.rpm
$ rpmbuild -ba -D "dist .fc41" SPECS/monpaquet.spec

```

```
[USER] rpmsort / rpmdumpheader / rpmquery / rpmverify
```

```

DESCRIPTION   : Outils utilitaires RPM complémentaires
POURQUOI      : Trier des listes de paquets RPM, inspecter les en-têtes, alias pratiques
                  de rpm -q et rpm -V.
CONTEXTE      : Scripts d'administration, packaging, audit.

```

```

rpmsort :
    Trie une liste de noms de paquets RPM selon les règles de version RPM
    $ rpm -qa | rpmsort
    $ ls *.rpm | rpmsort

rpmduheader :
    Affiche l'en-tête binaire d'un fichier RPM sous forme lisible
    $ rpmduheader paquet.rpm

rpmquery :
    Alias de rpm -q
    $ rpmquery vim

rpmverify :
    Alias de rpm -V
    $ rpmverify vim

```

```
[USER] makedeltarpm / applydeltarpm / combinedeltarpm
```

```

DESCRIPTION : Génère, applique et combine des paquets RPM différentiels (delta RPM)
POURQUOI    : Les delta RPM ne contiennent que les différences entre deux versions
              d'un paquet – réduit la bande passante lors des mises à jour
              (utilisé par DNF avec le plugin deltarpm).
CONTEXTE    : Mises à jour système optimisées, infrastructure de distribution RPM.

```

```

makedeltarpm :
    $ makedeltarpm ancien.rpm nouveau.rpm delta.drpm

applydeltarpm :
    $ applydeltarpm -r ancien.rpm delta.drpm nouveau.rpm

combinedeltarpm :
    Combine deux delta RPM en un seul (A→B + B→C = A→C)
    $ combinedeltarpm ancien.rpm delta_A_B.drpm delta_B_C.drpm delta_A_C.drpm

```

```
[USER] make / gmake
```

```

DESCRIPTION : Système de build automatisé basé sur des fichiers Makefile
POURQUOI    : Compiler des projets C/C++ et autres langages, automatiser des tâches
              répétitives. make détermine intelligemment quels fichiers recompiler
              selon les dépendances et les dates de modification.
CONTEXTE    : Compilation de logiciels, build de projets, automatisation.
OPTIONS     :
    -j N          Compilation parallèle sur N cœurs (-j sans N = max)
    -f FICHIER    Utilise ce Makefile (défaut : Makefile)
    -C RÉPERTOIRE Change de répertoire avant de lancer make
    -n / --dry-run Affiche les commandes sans les exécuter
    -k           Continue malgré les erreurs
    -s / --silent Mode silencieux
    -B / --always-make Recompile tout même si à jour
    -p           Affiche la base de données interne de make
    VARIABLE=VALEUR Définit une variable make
    CIBLE        Exécute cette cible spécifique
CIBLES CONVENTIONNELLES :
    all          Compile tout (cible par défaut)
    install      Installe les binaires compilés
    clean        Supprime les fichiers générés
    distclean    Suppression complète (inclut la config)
    check / test Lance les tests
    uninstall    Désinstalle
EXEMPLES      :
    $ make
    $ make -j$(nproc)
    $ make -j8
    $ sudo make install
    $ make clean && make -j$(nproc)
    $ make -n install
    $ make -C /chemin/vers/projet
    $ make CFLAGS="-O2 -march=native"

```

```
[USER] gcc / g++
```

```

DESCRIPTION : Compilateur C (gcc) et C++ (g++) de la GNU Compiler Collection
POURQUOI   : Compiler du code source C/C++ en binaires exécutables ou bibliothèques.
              Le compilateur de référence sous Linux.
CONTEXTE   : Développement, compilation de logiciels, optimisation de code.
OPTIONS PRINCIPALES :
  -o FICHIER      Nom du fichier de sortie
  -c              Compile seulement (produit un .o, sans linker)
  -S              Génère le code assembleur (.s)
  -E              Prétraitement seulement
  -O0 / -O1 / -O2 / -O3  Niveau d'optimisation (0=aucune, 3=maximale)
  -Os            Optimise pour la taille
  -Og            Optimise pour le débogage
  -g             Inclut les informations de débogage
  -Wall          Active la plupart des avertissements
  -Wextra        Avertissements supplémentaires
  -Werror        Transforme les avertissements en erreurs
  -I RÉPERTOIRE   Répertoire de headers supplémentaire
  -L RÉPERTOIRE   Répertoire de bibliothèques supplémentaire
  -l NOM         Lie avec la bibliothèque libNOM.so
  -shared        Crée une bibliothèque partagée (.so)
  -static        Lie statiquement
  -fPIC          Code indépendant de la position (pour .so)
  -D MACRO       Définit une macro préprocesseur
  -std=c11 / -std=c++17  Standard de langage à utiliser
  -march=native  Optimise pour le CPU actuel
  -pipe          Utilise des pipes entre les étapes (plus rapide)
  -v            Affiche les étapes de compilation

```

```

EXEMPLES :
$ gcc -o hello hello.c
$ gcc -O2 -Wall -o programme main.c utils.c
$ gcc -g -o programme_debug main.c
$ gcc -c module.c
$ gcc -shared -fPIC -o libmonlib.so module.o
$ gcc -o programme main.c -lm -lpthread
$ g++ -std=c++17 -O2 -o programme main.cpp
$ gcc -march=native -O3 -o optimise code.c

```

```
[USER]  cpp
```

```

DESCRIPTION : Préprocesseur C – traite les directives #include, #define, #ifdef
              avant la compilation
POURQUOI   : Prétraiter du code C/C++ manuellement – utile pour déboguer les macros
              et les inclusions.
CONTEXTE   : Débogage de macros, génération de code, développement.
OPTIONS    :
  -D MACRO       Définit une macro
  -U MACRO       Annule une macro
  -I RÉPERTOIRE  Répertoire de headers
  -P            Supprime les marqueurs de ligne
  -C            Conserve les commentaires

```

```

EXEMPLES :
$ cpp fichier.c
$ cpp -D DEBUG -I /usr/include fichier.c
$ gcc -E fichier.c | less

```

```
[USER]  as
```

```

DESCRIPTION : Assembleur GNU – convertit du code assembleur (.s) en fichiers objets (.o)
POURQUOI   : Compiler du code assembleur. Appelé automatiquement par gcc lors de
              la chaîne de compilation.
CONTEXTE   : Développement bas niveau, assembleur, compilation manuelle.
OPTIONS    :
  -o FICHIER      Fichier objet de sortie
  -g             Inclut les informations de débogage
  --32 / --64     Mode 32 ou 64 bits

```

```

EXEMPLES :
$ as -o module.o module.s
$ as --64 -o module.o module.s

```

```
[USER]  ld / ld.bfd / ld.lld / lld
```

DESCRIPTION : Éditeur de liens (linker) GNU – assemble des fichiers objets en exécutables ou bibliothèques. ld.lld est le linker LLVM – plus rapide que ld.bfd pour les gros projets.

POURQUOI : Lier des fichiers .o et des bibliothèques en un exécutable final. Appelé automatiquement par gcc.

CONTEXTE : Développement, compilation manuelle, optimisation du build.

OPTIONS :

- o FICHIER Fichier de sortie
- l NOM Lie avec libNOM
- L RÉPERTOIRE Répertoire de recherche des bibliothèques
- shared Crée une bibliothèque partagée
- static Édition de liens statique
- e SYMBOLE Point d'entrée du programme
- r Édition de liens partielle (relocatable)
- script=FICHIER Utilise ce script de linker
- Map=FICHIER Génère une map des symboles
- gc-sections Supprime les sections inutilisées
- rpath RÉPERTOIRE Chemin de recherche des .so à l'exécution

EXEMPLES :

```
$ ld -o programme main.o utils.o -lc
$ ld.lld -o programme main.o utils.o
$ ld -shared -o libmon.so module.o
```

[USER] ar

DESCRIPTION : Crée et gère des archives statiques (.a) – bibliothèques statiques

POURQUOI : Regrouper des fichiers objets (.o) en une bibliothèque statique (.a) qui peut être liée dans des programmes.

CONTEXTE : Développement de bibliothèques, packaging de modules objets.

OPTIONS :

- r Remplace ou ajoute des fichiers dans l'archive
- c Crée l'archive si elle n'existe pas
- s Crée/met à jour l'index des symboles
- t Liste le contenu de l'archive
- x Extrait les fichiers de l'archive
- d Supprime un fichier de l'archive
- v Mode verbeux

EXEMPLES :

```
$ ar rcs libmonlib.a module1.o module2.o
$ ar t libmonlib.a
$ ar x libmonlib.a
$ ar d libmonlib.a module_inutile.o
```

[USER] ranlib

DESCRIPTION : Génère ou met à jour l'index des symboles d'une archive statique (.a)

POURQUOI : Permet au linker de retrouver rapidement les symboles dans une bibliothèque statique. Équivalent à ar s.

CONTEXTE : Développement de bibliothèques statiques.

EXEMPLES :

```
$ ranlib libmonlib.a
```

[USER] nm

DESCRIPTION : Affiche les symboles d'un fichier objet, bibliothèque ou exécutable

POURQUOI : Inspecter les symboles définis ou référencés dans un binaire – identifier les fonctions exportées, les symboles non résolus, déboguer les erreurs de linking.

CONTEXTE : Développement, débogage de linking, audit de bibliothèques.

OPTIONS :

- u Affiche seulement les symboles non définis
- D / --dynamic Affiche les symboles dynamiques
- g Affiche seulement les symboles externes
- n Trie par adresse
- C / --demangle Démangle les noms C++ (lisible)
- defined-only Seulement les symboles définis
- A Préfixe chaque ligne avec le nom du fichier

TYPES DE SYMBOLES COURANTS :

- T Code (section .text)
- D Données initialisées (section .data)
- B Données non initialisées (section .bss)
- U Symbole non défini (externe)

```

R                               Données en lecture seule
W                               Symbole faible (weak)
EXEMPLES      :
$ nm programme
$ nm -C libmonlib.a
$ nm -u programme
$ nm -D /usr/lib64/libc.so.6 | grep malloc
$ nm --defined-only -C libmonlib.so

```

```
[USER]  objdump
```

```

DESCRIPTION : Affiche des informations sur des fichiers objets et exécutables ELF
              (désassemblage, en-têtes, sections, symboles...)
POURQUOI    : Analyser la structure d'un binaire – désassembler du code, inspecter
              les sections, comprendre l'organisation mémoire.
CONTEXTE    : Développement bas niveau, audit de binaires, débogage de build.
OPTIONS      :
  -d          Désassemble les sections de code
  -D          Désassemble toutes les sections
  -S          Entremêle le code source et l'assembleur (si debug)
  -h          Affiche les en-têtes de sections
  -f          Affiche les en-têtes de fichiers
  -t          Affiche la table des symboles
  -T          Affiche la table des symboles dynamiques
  -r          Affiche les informations de relocation
  -s          Affiche le contenu des sections (hex)
  -j SECTION Limite à cette section
  -M intel    Syntaxe Intel (au lieu d'AT&T)
  -C          Démangle les noms C++
EXEMPLES     :
$ objdump -d programme
$ objdump -d -M intel programme
$ objdump -S -d programme_avec_debug
$ objdump -h programme
$ objdump -t libmonlib.a
$ objdump -T /usr/lib64/libc.so.6 | grep malloc

```

```
[USER]  objcopy
```

```

DESCRIPTION : Copie et transforme des fichiers objets – extrait, supprime ou ajoute
              des sections
POURQUOI    : Extraire les symboles de debug, convertir des formats binaires,
              séparer les informations de debug d'un binaire de production.
CONTEXTE    : Packaging de debug symbols, développement embarqué, build.
OPTIONS      :
  --strip-debug      Supprime les informations de debug
  --strip-all        Supprime tous les symboles
  --only-keep-debug  Conserve seulement les infos de debug
  -O FORMAT          Format de sortie (binary, ihex, elf64-x86-64...)
  -j SECTION         Ne conserve que cette section
  -R SECTION         Supprime cette section
  --add-section NOM=FICHIER Ajoute une section depuis un fichier
  --add-gnu-debuglink=FICHIER Ajoute un lien vers un fichier de debug séparé
EXEMPLES     :
$ objcopy --strip-debug programme programme_stripped
$ objcopy --only-keep-debug programme programme.debug
$ objcopy --add-gnu-debuglink=programme.debug programme_stripped
$ objcopy -O binary programme programme.bin
$ objcopy -O ihex firmware.elf firmware.hex

```

```
[USER]  strip
```

```

DESCRIPTION : Supprime les symboles et sections de debug d'un binaire
POURQUOI    : Réduire la taille des binaires pour la production.
CONTEXTE    : Packaging, déploiement, réduction de taille des binaires.
OPTIONS      :
  -s / --strip-all      Supprime tous les symboles
  -g / --strip-debug    Supprime seulement les symboles de debug
  --strip-unnneeded      Supprime les symboles inutiles au linking
  -o FICHIER            Fichier de sortie (ne modifie pas l'original)
  -p                    Préserve les dates
EXEMPLES     :

```

```
$ strip programme
$ strip -g -o programme_stripped programme
$ strip --strip-unneeded /usr/bin/monprogramme
```

```
[USER] readelf
```

DESCRIPTION : Affiche des informations détaillées sur les fichiers au format ELF

POURQUOI : Inspecter finement la structure d'un binaire ELF – en-têtes, segments, sections, symboles, dépendances dynamiques.

CONTEXTE : Développement, débogage de linker, audit de binaires.

OPTIONS :

- h En-tête ELF
- l En-têtes de programme (segments)
- S En-têtes de sections
- s Table des symboles
- d Section dynamique (.dynamic)
- r Relocations
- n Notes (build-id, ABI...)
- a Tout afficher
- W Format large (pas de troncature)
- dyn-syms Symboles dynamiques

EXEMPLES :

```
$ readelf -h /bin/ls
$ readelf -d /usr/lib64/libc.so.6
$ readelf -s programme
$ readelf -S programme
$ readelf -a programme | grep NEEDED
```

```
[USER] size
```

DESCRIPTION : Affiche la taille des sections d'un fichier objet ou exécutable (text, data, bss)

POURQUOI : Voir rapidement la répartition mémoire d'un binaire – utile pour le développement embarqué et l'optimisation.

CONTEXTE : Développement embarqué, optimisation, vérification de taille.

OPTIONS :

- A Format SystemV (détaillé par section)
- B Format BSD (défaut – text/data/bss/total)
- d Affiche en décimal
- x Affiche en hexadécimal

EXEMPLES :

```
$ size programme
$ size -A programme
$ size /usr/bin/vim /usr/bin/nano
```

```
[USER] ldd
```

DESCRIPTION : Affiche les bibliothèques partagées requises par un programme

POURQUOI : Identifier les dépendances dynamiques (.so) d'un exécutable – diagnostiquer les erreurs "library not found", vérifier les dépendances avant déploiement ou packaging.

CONTEXTE : Débogage de dépendances, packaging, déploiement.

OPTIONS :

- v Mode verbeux (toutes les dépendances récursives)
- u Affiche les dépendances inutilisées

AVERTISSEMENT : Ne jamais exécuter ldd sur un binaire non fiable – ldd peut exécuter du code. Utiliser readelf -d à la place pour les binaires suspects.

EXEMPLES :

```
$ ldd /usr/bin/vim
$ ldd /bin/ls
$ ldd -v /usr/lib64/libssl.so.3
```

```
[USER] strings
```

DESCRIPTION : Extrait les chaînes de caractères lisibles d'un fichier binaire

POURQUOI : Inspecter rapidement le contenu d'un binaire – trouver des messages, URLs, chemins, informations de version sans désassembler.

CONTEXTE : Audit de binaires, vérification de build, packaging.

OPTIONS :

- a / --all Analyse tout le fichier
- n N / --bytes=N Longueur minimale des chaînes (défaut : 4)


```
-t FORMAT          Affiche le décalage (o=octal, x=hex, d=décimal)
-f                Affiche le nom du fichier avant chaque chaîne

EXEMPLES          :
$ strings /usr/bin/vim
$ strings -n 8 programme
$ strings -t x programme | grep "http"
```

```
[USER]  c++filt
```

```
DESCRIPTION      : Démangle les noms de symboles C++ et Java
POURQUOI         : Les compilateurs C++ transforment les noms de fonctions (name mangling)
                  pour encoder la signature – c++filt les rend lisibles.
CONTEXTE         : Débogage de linking C++, lecture de sorties nm/objdump.
EXEMPLES         :
$ c++filt _ZNSt6vectorIiSaIiEE9push_backERKi
$ nm programme | c++filt
$ objdump -t programme | c++filt
```

```
[USER]  addr2line
```

```
DESCRIPTION      : Convertit des adresses mémoire en noms de fichiers source et numéros
                  de lignes (nécessite les infos de debug -g)
POURQUOI         : Identifier la ligne de code source correspondant à une adresse dans
                  un crash ou un stack trace.
CONTEXTE         : Débogage de crashes, analyse de core dumps, build.
OPTIONS          :
-e FICHIER        Fichier exécutable
-f               Affiche aussi le nom de la fonction
-C               Démangle les noms C++
-s               Affiche seulement le nom de fichier (sans le chemin)
-i               Remonte les fonctions inlinees
EXEMPLES         :
$ addr2line -e programme -f 0x4005a3
$ addr2line -e programme -fCi 0x4005a3
```

```
[USER]  pkg-config / pkgconf
```

```
DESCRIPTION      : Fournit les flags de compilation et de linking pour les bibliothèques
                  installées sur le système
POURQUOI         : Obtenir automatiquement les options -I, -L et -l nécessaires pour
                  compiler avec une bibliothèque – évite de les chercher manuellement.
CONTEXTE         : Compilation de logiciels utilisant des bibliothèques tierces.
OPTIONS          :
--cflags NOM      Drapeaux de compilation (include paths)
--libs NOM        Drapeaux de linking (bibliothèques)
--cflags --libs NOM Les deux ensemble
--modversion NOM  Version de la bibliothèque
--exists NOM      Vérifie si la bibliothèque est disponible
--list-all       Liste toutes les bibliothèques disponibles
--variable=VAR NOM Valeur d'une variable du fichier .pc
EXEMPLES         :
$ pkg-config --cflags --libs openssl
$ pkg-config --modversion glib-2.0
$ gcc -o programme main.c $(pkg-config --cflags --libs gtk4)
$ pkg-config --exists libcurl && echo "libcurl disponible"
$ pkg-config --list-all | grep ssl
```